Development Setup Guide

Vision-Based Illegal Waste Detection Using Geotagged Smartphone Images and YOLOv11

This guide sets up everything you need across your two devices:

- **iPhone 15 Pro Max** — your data collection device (takes the geotagged photos)
- **Windows laptop** — your development machine (code, model training/testing, GIS mapping, backend)

Follow the steps in order. Each step says exactly what to install and how.

Part 1 — iPhone 15 Pro Max: capturing geotagged photos

You don't need any special app for this. The built-in Camera app already embeds GPS coordinates and a timestamp into every photo's EXIF metadata, as long as Location Services is enabled for the Camera app.

1.1 Turn on location tagging

1. Open **Settings** → **Privacy & Security** → **Location Services**.
2. Make sure **Location Services** is toggled ON at the top.
3. Scroll down to **Camera**, tap it, and select **While Using the App**.

1.2 Turn on high-accuracy location (optional but recommended)

1. Settings → Privacy & Security → Location Services → System Services.
2. Make sure **Precise Location** is enabled for the Camera (this is usually on by default on iOS 17+).

1.3 Take your photos

- Use the native Camera app in standard **Photo** mode (not Live Photo, not Portrait — plain photos keep EXIF simplest to parse).
- Walk around your target sites in Harare/Bulawayo: illegal dumps, roadsides, clean areas, bins, rivers, markets, residential areas — matching your dataset classes.
- Try to vary lighting, distance, and angle for each waste type — this directly improves model generalisation.

1.4 Get the photos onto your Windows laptop

You have three good options; pick whichever is easiest for you:

- **Option A — USB cable (most reliable, keeps EXIF intact):**

1. Connect the iPhone to the laptop with a USB-C to Lightning (or USB-C to USB-C, if you have the newer cable) cable.
2. On the iPhone, tap **Trust This Computer** when prompted.
3. On Windows, open **File Explorer** → **This PC** → your iPhone should appear as a device.
4. Navigate to `Internal Storage\DCIM` and copy the photos to a folder on your laptop, e.g. `C:\WasteProject\raw_images`.

- **Option B — iCloud Photos + iCloud for Windows:**
  1. Install **iCloud for Windows** from the Microsoft Store.
  2. Sign in with your Apple ID and enable **iCloud Photos**.
  3. Your photos sync automatically to `C:\Users\<you>\iCloudPhotos`.
  4. ⚠️ Note: iCloud Photos can sometimes strip or alter EXIF data depending on settings — after transferring a test batch, check a sample image's metadata (Part 2.6 below) before doing a full transfer.
- **Option C — Google Photos app (free, keeps EXIF):**
  1. Install the **Google Photos** app from the App Store on your iPhone.
  2. Turn on backup with **Original quality**.
  3. On your laptop, go to photos.google.com and download the images.

Recommendation: Use Option A (USB) for your main dataset — it's the most reliable way to guarantee GPS/timestamp metadata survives the transfer.

Part 2 — Windows laptop: core development environment

2.1 Check your hardware first

Open **Task Manager** → **Performance** tab → check if you have a dedicated **NVIDIA GPU**. This determines your training strategy:

- **No dedicated NVIDIA GPU** (most laptops, including anything with only Intel/AMD integrated graphics): train on **Google Colab's free GPU tier** (Part 4) and use your laptop only for coding, data prep, and running the trained model for inference/testing. This matches your proposal's budget assumptions.
- **Dedicated NVIDIA GPU**: you can optionally train locally too — see the note in 2.5.

2.2 Install Python

1. Go to **python.org/downloads** and download **Python 3.11** (3.11 has the best current compatibility with PyTorch/Ultralytics; avoid 3.13 for now).
2. Run the installer.
3. ⚠️ **Critical step:** On the first installer screen, tick the checkbox **"Add python.exe to PATH"** at the bottom before clicking Install.

4. Click **Install Now**.
5. Verify it worked — open **Command Prompt** (search "cmd" in the Start menu) and run:

```
python --version
pip --version
```

You should see version numbers, not an error.

2.3 Install Git for Windows

1. Download from **git-scm.com/download/win**.
2. Run the installer — the default options are fine for all screens; just keep clicking **Next** then **Install**.
3. Verify:

```
git --version
```

2.4 Install Visual Studio Code

1. Download from **code.visualstudio.com**.
2. Run the installer. On the "Select Additional Tasks" screen, tick **"Add to PATH"** and **"Register Code as an editor for supported file types"**.
3. Open VS Code, go to the **Extensions** panel (left sidebar, square icon), and install:
   - **Python** (by Microsoft)
   - **Pylance** (by Microsoft)
   - **Jupyter** (by Microsoft) — lets you run notebook-style cells, useful for experimenting with the model

2.5 Create your project folder and a virtual environment

A virtual environment keeps this project's Python packages separate from everything else on your system — always do this.

1. Open **Command Prompt** and create your project structure:

```
mkdir C:\WasteProject
cd C:\WasteProject
mkdir raw_images dataset scripts models notebooks app
```

2. Create the virtual environment:

```
python -m venv venv
```

3. Activate it (you'll need to do this every time you open a new terminal to work on the project):

```
venv\Scripts\activate
```

Your prompt should now show `(venv)` at the start of the line. 4. Open the folder in VS Code:

```
code .
```

In VS Code, press `Ctrl+Shift+P` → type "Python: Select Interpreter" → choose the one inside `.\venv\`.

2.6 Install the core Python packages

With `(venv)` active in your terminal, run:

```
pip install ultralytics opencv-python numpy pandas matplotlib pillow exifread
```

What each one does:

- **ultralytics** — the official YOLOv11 package (training, inference, export)
- **opencv-python** — image loading/processing
- **numpy** / **pandas** — data handling
- **matplotlib** — plotting training curves and results
- **pillow** — image manipulation (used alongside OpenCV)
- **exifread** — reads GPS/timestamp metadata out of your iPhone photos

Note on PyTorch: `ultralytics` installs a CPU version of PyTorch automatically. That's fine for testing code and running inference on single images. If you later find you have a dedicated NVIDIA GPU and want to train locally, install the GPU version instead by first checking your CUDA version (`nvidia-smi` in Command Prompt) and then following the exact command generator at **pytorch.org/get-started/locally** — pick Windows, Pip, Python, and your CUDA version, then run the command it gives you.

Quick test that everything works:

```
python -c "from ultralytics import YOLO; m = YOLO('yolo11n.pt');
print('YOLOv11 loaded successfully')"
```

This will download the small pretrained YOLOv11n model (~5 MB) and confirm your setup is correct.

Check that an iPhone photo's GPS data is readable:

```
python -c "import exifread;
f=open(r'C:\WasteProject\raw_images\test.jpg','rb');
tags=exifread.process_file(f); print(tags.get('GPS GPSLatitude'),
tags.get('GPS GPSLongitude'))"
```

(Replace `test.jpg` with an actual filename from your transferred photos.) If this prints coordinate values instead of `None None`, your geotagging pipeline is working end to end.

Part 3 — Annotation tool: Roboflow

Roboflow is the easiest annotation tool to start with — it's entirely browser-based, has a generous free tier, and exports directly in YOLO format, so no separate install is needed.

1. Go to **roboflow.com** and sign up for a free account.
2. Create a new project → choose **Object Detection**.
3. Upload your transferred iPhone photos.
4. Draw bounding boxes around waste items and label them with your classes (Plastic, Organic Waste, Metal, Tyres, Construction Waste, Mixed Waste).
5. Once you've annotated enough images, use Roboflow's **Generate** → **Export Dataset** → format **YOLOv11** (or "YOLOv8 PyTorch TXT", which is compatible) to get a ready-to-train dataset folder.

Part 4 — Google Colab: free GPU training

1. Go to **colab.research.google.com** and sign in with a Google account (create one if you don't have one).
2. Create a new notebook.
3. Go to **Runtime** → **Change runtime type** → set **Hardware accelerator** to **GPU** (T4 is fine on the free tier).
4. In the first cell, install Ultralytics:

```
python

   !pip install ultralytics
```

5. Upload your Roboflow-exported dataset (either drag-and-drop into Colab's file panel, or mount Google Drive) and start training with a command like:

```
python
```

```
from ultralytics import YOLO
model = YOLO("yolo11n.pt")
model.train(data="data.yaml", epochs=100, imgsz=640)
```

6. Download the resulting `best.pt` weights file back to your laptop's `models` folder when training finishes.

Part 5 — GIS and mapping tools

QGIS (desktop GIS software):

1. Download from **qgis.org/download** → choose the Windows installer.
2. Run it with default options.

Folium (Python mapping library): With `(venv)` active:

```
pip install folium
```

No further setup needed — you'll use this to plot detection coordinates on an interactive map from Python.

Part 6 — Backend and database

With `(venv)` active, install the web framework and database driver:

```
pip install flask flask-sqlalchemy
```

- **Flask** — lightweight web framework for the reporting backend
- **flask-sqlalchemy** — makes it easy to work with a database from Flask

Database: SQLite needs no installation — it's built into Python's standard library (`sqlite3`) and Flask-SQLAlchemy will create the database file automatically the first time you run the app. You only need to install PostgreSQL separately later if you outgrow SQLite (unlikely for this project's scope).

Part 7 — Version control with Git and GitHub

1. Go to **github.com** and create a free account if you don't have one.
2. Create a new repository, e.g. `waste-detection-zimbabwe`.
3. Back in Command Prompt, inside `C:\WasteProject`:

```
   git init
   git add .
   git commit -m "Initial project structure"
   git branch -M main
   git remote add origin https://github.com/<your-username>/waste-detection-
zimbabwe.git
   git push -u origin main
```

4. Create a `.gitignore` file in `C:\WasteProject` to keep large files and the virtual environment out of GitHub:

```
   venv/
   raw_images/
   dataset/
   *.pt
   __pycache__/
```

Suggested project folder structure

```
C:\WasteProject
├── venv/                # virtual environment (not committed to Git)
├── raw_images/           # unannotated iPhone photos
├── dataset/              # Roboflow-exported annotated dataset
├── models/               # trained YOLOv11 weight files (best.pt)
├── notebooks/            # experimentation notebooks
├── scripts/              # standalone Python scripts (e.g. exif_extract.py,
detect.py)
├── app/                  # Flask backend + mapping code
├── .gitignore
└── README.md
```

What's next

Once everything above is installed and the two quick tests in Part 2.6 pass, the natural next step is:

1. Collect and annotate your first batch of images (aim for at least a few hundred to start training meaningfully).
2. Run a first baseline training pass on Colab, even with a small dataset, just to confirm the whole pipeline works end to end.

3. Then iterate: add more images, refine classes, and retrain.

Let me know once you've got Python, Git, and VS Code installed and I can help you write the first script — extracting GPS/timestamp data from your iPhone photos is a good starting point.